Relazione progetto Connections

Lab Reti 3

21 maggio 2026

1. Scelte progettuali

Il progetto realizza una versione client-server del gioco *Connections*. La scelta principale è stata mantenere una sola partita globale attiva alla volta, gestita centralmente dal server: tutti gli utenti loggati partecipano alla stessa partita corrente e, allo scadere del tempo, il server archivia il risultato e avvia automaticamente la partita successiva.

La comunicazione ordinaria client-server avviene su una connessione TCP persistente con messaggi JSON terminati da `\n`. Questa scelta semplifica il parsing dei messaggi su canali NIO non bloccanti: il buffer può contenere messaggi parziali o multipli e `MessageReader.extractMessage` estrae solo quelli completi. Le notifiche asincrone di fine partita sono invece inviate via UDP, in modo che il server possa avvisare i client senza attendere una richiesta esplicita.

La registrazione, il login, l'aggiornamento credenziali, le proposte di gioco, le richieste di stato, statistiche e classifica sono implementate come operazioni JSON con campo `operation`. I dati persistenti sono salvati in file JSON: utenti in `data/users.json`, cronologia partite in `data/gameHistory.json`, puzzle in `data/Connections_Data.json`. La persistenza avviene periodicamente e anche alla chiusura del server tramite shutdown hook.

Per il punteggio si applicano le regole del progetto: ogni gruppo corretto dà bonus progressivo di 6 punti, ogni proposta errata sottrae 4 punti, e il quarto gruppo viene considerato implicito quando il giocatore trova i primi tre. Le proposte malformate, ad esempio parole duplicate, parole non presenti o parole già assegnate, vengono rifiutate senza modificare lo stato della partita.

2. Schema dei thread

Lato server

Thread / componente	Funzione
Thread principale `ServerMain`	Apre il `Selector`, accetta connessioni TCP, legge messaggi dai client e crea i task di gestione richiesta.
Thread pool fisso	Esegue le richieste applicative dei client: registrazione, login, invio proposta, statistiche, classifica, ecc. È creato con `Executors.newFixedThreadPool(10)`.
Scheduler salvataggi	Esegue periodicamente `userManager.save()` e `gameManager.saveHistory()` secondo `persistence.interval.seconds`.
Scheduler partita	In `GameManager` pianifica la fine della partita con `ScheduledExecutorService`; allo scadere chiama `onGameEnd()`.
Shutdown hook	Salva utenti e cronologia prima della terminazione del processo server.

Schema logico:

```
ServerMain / Selector TCP
    |-- accetta client e legge JSON
    |-- ThreadPool(10)
          |-- UserManager
          |-- GameManager
          |-- UdpNotifier
    |-- Scheduler salvataggi
    |-- Scheduler fine partita
```

Lato client

Thread / componente	Funzione
Thread principale `ClientMain`	Gestisce la connessione TCP NIO verso il server, legge risposte e invia i messaggi presenti nella coda.
`CliThread`	Legge i comandi da terminale, costruisce richieste JSON e le inserisce nella coda condivisa.
`UdpListener`	Resta in ascolto su una porta UDP scelta automaticamente e stampa le notifiche asincrone di fine partita. È avviato come daemon.

Schema logico:

```
ClientMain / Selector TCP
    |-- legge risposte dal server
    |-- invia messaggi dalla coda
CliThread
    |-- input utente -> JSON -> coda
UdpListener
    |-- notifiche UDP di fine partita
```

3. Strutture dati

Server

`ClientSession` rappresenta lo stato della connessione TCP di un client: buffer di lettura, username loggato, porta UDP e indirizzo IP del client.

`UserManager` mantiene:

- `ConcurrentHashMap<String, UserData> users`, mappa utenti registrati;
- `Set<String> loggedIn`, insieme thread-safe degli utenti loggati;
- `Set<ClientSession> activeSession`, sessioni attive usate per le notifiche UDP.

`UserData` contiene username, password, punteggio totale, partite giocate, vittorie, streak corrente, streak massimo, perfect puzzle e istogramma degli errori.

`GameManager` mantiene:

- `currentPuzzle`, puzzle globale attivo;
- `gameEndTime`, istante di fine partita;
- `ConcurrentHashMap<String, PlayerGameState> playerState`, stato di gioco per ciascun utente;
- `List<GameRecord> gameHistory`, cronologia delle partite concluse;
- riferimenti a `PuzzleRepository`, `UserManager` e `UdpNotifier`.

`PlayerGameState` descrive la partita del singolo giocatore: numero errori, gruppi corretti, punteggio, flag `finished` e flag `online`.

`PuzzleRepository` carica i puzzle da JSON e mantiene l'indice del prossimo puzzle. `GameRecord` salva l'esito storico di una partita: id, soluzione, stati finali dei giocatori, numero conclusi, vincitori e punteggio medio.

Client

Il client usa una `ConcurrentLinkedQueue<String>` per passare messaggi JSON dal thread CLI al thread NIO. Usa inoltre un `ByteBuffer` di lettura per le risposte TCP e un eventuale `currentWriteBuffer` per gestire scritture parziali.

Le variabili `volatile ClientMain.isLoggedIn` e `ClientMain.waitingForResponse` coordinano lo stato dell'interfaccia: la prima indica se mostrare il menu iniziale o quello di gioco, la seconda blocca temporaneamente la CLI finché arriva la risposta del server.

4. Sincronizzazione

Sul server, le strutture condivise sono protette combinando collezioni concorrenti e metodi `synchronized`.

In `UserManager` sono sincronizzati `load`, `save`, `register`, `login`, `logout`, `recordGameResult`, `updateCredentials` e `getLoggedUsers`. Questo evita aggiornamenti incoerenti su utenti, login e statistiche quando più richieste sono eseguite

in parallelo dal thread pool. Le collezioni `ConcurrentHashMap` e `ConcurrentHashMap.newKeySet()` permettono inoltre accessi concorrenti sicuri per letture e inserimenti.

In `GameManager` sono sincronizzate le operazioni critiche sul ciclo della partita: `startNewGame`, `onGameEnd`, `submitProposal`, `getNextGameId`, `loadHistory`, `saveHistory` e `leaveCurrentGame`. In questo modo una proposta non può sovrapporsi in modo incoerente alla chiusura della partita o all'avvio della successiva. `currentPuzzle` e `gameEndTime` sono `volatile`, così i thread che leggono stato partita vedono il valore più aggiornato.

In `PuzzleRepository`, `getNextPuzzle` e `setCurrentIndex` sono sincronizzati per rendere atomico l'avanzamento dell'indice dei puzzle.

In `JsonPersistence.writeJson` il metodo è `synchronized` e scrive prima su file temporaneo `.tmp`, poi rinomina il file: questo riduce il rischio di lasciare un JSON parzialmente scritto in caso di errore durante la scrittura.

Sul client, la coda `ConcurrentLinkedQueue` sincronizza il passaggio dei messaggi tra CLI e thread NIO. Le variabili `volatile` evitano che il thread CLI legga valori obsoleti di login o attesa risposta. Il `Selector.wakeup()` viene chiamato dopo l'inserimento in coda per risvegliare il thread NIO e fargli registrare l'interesse alla scrittura.

5. Manuale di compilazione ed esecuzione

Requisiti

- Java JDK installato, con `javac` e `java` disponibili da terminale.
- Libreria esterna Gson già presente nel progetto: `lib/gson-2.13.2.jar`.
- Avviare i comandi dalla cartella radice del progetto.

File di configurazione

Server: `config/server.properties`

```
tcp.port=8080
udp.port=8081
game.duration.seconds=300
persistence.interval.seconds=30
users.file=data/users.json
```

Client: `config/client.properties`

```
server.ip=localhost
server.tcp.port=8080
```

Per test veloci si può ridurre `game.duration.seconds`, ad esempio a `60`. Se server e client sono su macchine diverse, nel client impostare `server.ip` con l'indirizzo IP del server.

Compilazione

Da macOS/Linux:

```
javac -cp "lib/gson-2.13.2.jar:src" -d out $(find src -name "*.java")
```

Da Windows PowerShell:

```
javac -cp "lib/gson-2.13.2.jar;src" -d out (Get-ChildItem -Recurse src -Filter *.java).FullName
```

Avvio del server

Da macOS/Linux:

```
java -cp "out:lib/gson-2.13.2.jar" server.ServerMain
```

Da Windows:

```
java -cp "out;lib/gson-2.13.2.jar" server.ServerMain
```

Il server apre la porta TCP configurata, di default `8080`, e usa la porta UDP configurata, di default `8081`, per inviare notifiche.

Avvio di uno o più client

Aprire un nuovo terminale per ogni client.

Da macOS/Linux:

```
java -cp "out:lib/gson-2.13.2.jar" client.ClientMain
```

Da Windows:

```
java -cp "out;lib/gson-2.13.2.jar" client.ClientMain
```

All'avvio il client mostra il menu iniziale:

- `1` registra un nuovo utente;
- `2` effettua il login;
- `0` chiude il client.

Dopo il login il client entra automaticamente nella partita corrente e mostra il menu di gioco:

- `1` invia una proposta di 4 parole;
- `2` richiede info partita corrente o storica;
- `3` richiede statistiche partita corrente o storica;
- `4` mostra statistiche personali;
- `5` mostra classifica completa, top K o posizione di un giocatore;
- `6` aggiorna username e/o password;
- `0` effettua logout.

Le notifiche di fine partita arrivano automaticamente via UDP e mostrano statistiche globali e soluzione completa. I file `data/users.json` e `data/gameHistory.json` vengono aggiornati periodicamente e alla chiusura del server.